

技术文档 · TECHNICAL DOCUMENT

Haven-Store

二手商品交易平台的 AI 辅助开发技术文档（v2.0 扩写版）：系统阐述本次全方位升级的意图控制策略（如何设计上下文与 Prompt）、架构设计（AI 生成代码采用的架构与约束手段）与可复现性论证。优化清单共 56 项、落地 52 项；正文含性能基准实测、端到端案例详解与 336 项冒烟断言、66 项单元测试全绿的可复现基线。

版本 2.0 · 2026 年 10 月

目录

本文档面向三类读者：项目评审者、后续接手开发的工程师、负责部署与运维的人员。全文围绕 2026 年 10 月完成的一次系统性升级展开，回答三个问题：AI 辅助开发中如何控制生成意图（第二章）、升级后的代码采用什么架构以及如何约束（第三章）、成果如何被第三方复现与验证（第四章）。第五章给出结论，附录 A~H 提供环境变量、接口清单、故障处置、配置样例、测试用例分布、术语表、端到端案例与参考资料。

编号约定：升级过程中的 56 项优化条目按 A~F 六组管理——A 必修缺陷（10 项）、B 普通用户端（12 项）、C 管理员端（9 项）、D 前端体验（10 项）、E 安全与健壮性（10 项）、F 工程化（5 项）；正文引用形如「A2」「C8」。图表按章编号：表 2-1 表示第 2 章第 1 张表，图、清单同理。

表 0-1 全文章节导览

章节	内容
第一章 项目概述	背景与问题定义、技术栈选型对比、升级范围与前后对比
第二章 意图控制策略	协作流程、上下文设计与实测成本、Prompt 原则与模式、阶段验证实录、局限性
第三章 架构设计	总体架构、分层依赖、数据模型与索引、契约层、鉴权链路、并发一致性、前端架构、接口规范、约束手段
第四章 可复现性论证	复现性声明、依赖锁定、配置即代码、行为断言分布、性能基准实测、非确定性、演练记录
第五章 总结与展望	成效数据与后续路线
附录 A~C	环境变量表·接口清单摘要·常见问题
附录 D~H	配置样例·测试用例分布·术语表·端到端案例·参考资料

第一章 项目概述

1.1 项目背景与升级范围

Haven-Store 是一个基于 Spring Boot 与 Vue 3 的全栈二手商品交易平台，提供商品发布、附近商品检索、AI 估价、商品评价、收藏与购物车、下单支付（模拟支付）、私信与商品咨询、系统消息、管理后台等能力。项目代码仓库为 github.com/istilljump/Haven-Store。

「待补充：项目立项背景（课程设计、竞赛或产品原型）与目标用户规模。」

升级启动前，项目处于「功能骨架完整、工程质量欠账」的状态。两个只读探索代理的全量盘点将问题归为三类：其一，安全隐患——存在一个任何登录用户可伪造系统公告的权限漏洞，禁用账号在 Token 有效期内仍可正常交易，登录接口无任何防爆破措施；其二，未生效功能——多个界面元素背后没有对应的真实实现（本文件统一称为「未生效功能」，例如行内排序只改本地数组不调接口、系统设置保存后对后端行为零影响）；其三，能力缺口——对照一个可日常使用的二手交易平台，缺少通知中心、订单详情、举报处理、数据看板等必要模块。

2026 年 10 月，项目完成了一次系统性升级。优化清单共 56 项，按 A~F 六组编号（见目录页约定），经人工逐条审查后分五个阶段实施；本轮落地 52 项，其余 4 项按审查约定延后或调整：B12（浏览历史）、C9（操作日志）、E10（手机号全面脱敏）为可选项未实施，F3（统一分页 VO）因涉及全部列表接口的契约变更，评估后暂缓。

1.2 技术栈与选型对比

技术栈以「课设/小团队可维护」为第一原则：框架成熟、文档充分、社区答案多。关键选型的备选方案与取舍理由如下：

表 1-1 关键选型对比

决策点	采用方案	备选方案	取舍理由
后端框架	Spring Boot 2.7.12	Spring Boot 3.x	3.x 需 JDK 17 与 javax→jakarta 迁移，存量代码改造成本高于收益；2.7 仍在维护窗口内
ORM	MyBatis-Plus 3.5.3.1	Spring Data JPA	项目查询以多条件动态筛选为主，Wrapper 风格可控性更强；团队已有 MyBatis 使用经验
UI 组件库	Element Plus ^2.4	Ant Design Vue	表单/表格/上传组件覆盖管理后台场景更全，中文文档与示例更丰富
实时消息	HTTP 轮询（10s/30s）	WebSocket	校园局域网规模下轮询足够；WebSocket 需要会话管理与网关配合，列为后续演进项
密码散列	BCrypt (spring-security-crypto)	MD5/SHA-256 + 盐	BCrypt 自带盐与可调用代价因子，抗暴力破解；MD5 已被证明不适用于口令存储
数据可视化	Apache ECharts ^5.5	Chart.js	中文文档完善，饼图/折线图交互能力强；按需分包后仅管理端加载

完整依赖清单见附录 D 的 pom.xml 关键段。运行时要求：JDK 8+（推荐 17/22）、Node.js 18+、MySQL 8.0+、Redis 7+；全部依赖均可离线获取，无云服务依赖。

1.3 交付成果概览

56 项清单条目按六组归类，各组交付情况如下：

表 1-2 交付成果分组统计

分组	数量	代表性成果
A 必修缺陷	10 项	注册功能打通；系统消息伪造漏洞摘除；禁用账号 Token 立即失效；分类排序未保存、设置页无法保存等未生效功能修复
B 普通用户端	12 项	首页重构、通知中心、订单详情页、我的发布管理页、AI 估价入口、附近商品、评价与订单关联、商品举报、搜索排序增强、订单超时自动取消
C 管理员端	9 项	订单管理（含导出）、评论管理、举报处理台、用户编辑与重置密码、ECharts 数据看板、系统设置落库并真实生效、群发消息批次化
D 前端体验	10 项	共享组件（ApplImage/ProductCard）、全局请求进度条、favicon、移动端适配收口、死代码清理
E 安全与健壮性	10 项	登录防爆破、上传魔数校验、事务补齐、Redis 序列化白名单、CSV 公式注入防护、条件更新消除并发重复推进
F 工程化	5 项	分类缓存、评分改 SQL 聚合、ESLint/Prettier 配置补齐、依赖显式声明、测试回归（F3 统一分页 VO 暂缓）

1.4 升级前后对比

表 1-3 升级前后对比

维度	升级前	升级后
安全	/message/send 端点任何登录用户可伪造系统公告；禁用账号的 Token 在有效期内仍可交易	漏洞端点移除；拦截器逐请求校验账号状态，禁用即时失效；登录锁定与密码策略生效
用户端	注册页未实现提交逻辑；首页无商品内容；无通知中心、订单详情、我的发布页	注册-发布-搜索-附近-估价-下单-支付-评价-举报-私信-通知全链路可用
管理端	无法查看订单与评论；无举报处理；系统设置保存后对后端行为无影响	订单/评论/举报/用户管理/看板图表齐备；设置落库并真实约束业务
测试	冒烟测试 26 项断言失败；单元测试 66 项中 63 项从未通过	冒烟测试 336 项断言全部通过；单元测试 66 项全部通过
工程	无 ESLint 配置；图标库为未声明的幽灵依赖；存在多个死代码文件	Lint 配置补齐；依赖显式声明；死代码清零；ECharts 独立分包

第二章 意图控制策略

AI 辅助开发的主要风险不是「写不出代码」，而是「写出的代码与意图偏离」：擅自扩大范围、引入未经审查的方案、编造不存在的接口。本章记录升级过程中控制这类风险的三个手段：协作流程设计（2.1）、上下文工程（2.2）与 Prompt 约束（2.3~2.4）；2.5 给出各阶段的验证闸门执行实录；2.6 说明发生偏离后的纠偏机制；2.7 讨论该方法的适用边界。

2.1 总体流程：先对齐、再实施、逐步验证

「生成代码」在整条流水线中占时最短；多数工作量花在生成之前的事实对齐与生成之后的验证上：

图 2-1 AI 辅助升级的六步协作流程

- ① 全量盘点：两个只读探索代理并行扫描前后端，产出结构化现状报告

② 清单化：将盘点结论整理为 56 项优化条目，按 A~F 分组编号并标注优先级

③ 人工审查：逐条确认范围，可选项显式二选一（如 C8 商品审核流选择「轻量方案」）

④ 排序实施：按 A → E → B → C → D/F 的顺序建立贯穿全程的任务清单

⑤ 分阶段验证：每个阶段以「编译零错误 + 冒烟测试不出现新失败」作为完成定义

⑥ 全量回归：单元测试 66 项 + 冒烟测试 336 项 + 前端构建

这一流程的核心作用是：把一句「全方位升级」转化为 56 项带编号、可追溯、可验证的具体条目；任何一处代码改动都能对应回某个清单编号，AI 不存在绕过清单自由发挥的空间。

2.2 上下文设计

2.2.1 结构化盘点先行

在编写任何升级代码之前，两个只读探索代理（AI 编码工具中承担调研任务的子代理，只读不写）分别对后端约 50 个接口与前端 20 个视图做全量扫描，产出包含接口清单、各域业务现状、安全与健壮性问题（附文件与行号）、配置审查、缺失功能对照在内的结构化报告。这份报告是最重要的上下文投入：后续每次代码生成都在同一份事实基础上进行。其直接收益是避免了凭印象编码——例如盘点发现前端 user.js 中封装了 10 个后端并不存在的接口，这类问题靠逐页浏览很难被发现。

盘点的实测成本见表 2-1：两个代理合计消耗约 336 万 token、140 次工具调用、用时约 23 分钟。相对于后续数天的编码与调试，这一投入占比很小，却决定了后续所有生成的质量基线。

表 2-1 盘点阶段实测成本（两个只读代理并行）

代理	扫描对象	token 消耗	工具调用次数	用时
代理一（后端）	约 50 个端点、9 个 Controller、配置与建表脚本	约 182.7 万 token	53 次	约 11 分钟
代理二（前端）	20 个视图、6 个 API 封装、路由与状态管理	约 153.0 万 token	87 次	约 12 分钟
合计	—	约 335.7 万 token	140 次	约 23 分钟（并行）

2.2.2 上下文分级加载

并非所有信息都需要常驻上下文。本项目按四个层级管理，以控制上下文窗口的信噪比：

表 2-2 上下文分级加载策略

层级	内容	加载时机	示例
需求级	经审查的 56 项清单与裁剪决策	全程常驻	C8 选择轻量方案；本轮不接入 WebSocket
架构级	技术栈、分层约定、统一返回结构、鉴权链路	进入某模块前	Result{code,msg,data} 契约、按域分包模式
局部级	待修改文件全文及其关联文件	每次编辑前	修改 ProductServiceImpl 前先通读全部约 900 行
事实级	数据库 Schema、枚举取值、接口路径	涉及契约时	商品状态仅 1/2/3，订单状态仅 1~4

2.2.3 任务清单作为持久锚点

实施全程维护一份与清单编号对应的任务清单，完成一项即勾选一项。它的作用超出进度管理：当会话上下文因超长被摘要压缩时，任务清单是唯一跨压缩边界保留的总纲，保证压缩后的生成仍然锚定在已审查的范围内。

2.3 Prompt 设计原则

表 2-3 Prompt 设计五原则

原则	做法	反例
验收标准前置	每阶段开始前先声明「完成的定义」：编译零错误、冒烟测试无新增失败	先写代码，再考虑如何验证
约束优先于指令	显式列出不做的事项及理由：真实支付网关、WebSocket 推送、Spring Boot 3 迁移、暗色模式	只描述要做什么，边界由模型自行决定
对齐既有契约	新代码的字段名、枚举值、状态码必须引用既有常量（OrderStatusEnum、ProductStatusEnum、ResultCodeEnum），禁止魔法值	前后端各自维护一套硬编码状态码
决策显式化	可选项要求「二选一并说明理由」，结论回写清单与文档	在多个方案之间默认折中，不留决策记录
小步交付	每个阶段结束给出可验证节点，而非一次性交付全部改动	全部完成后再统一验证

实践中收益最大的是「对齐既有契约」：本轮冒烟测试出现的 26 项失败，多数源于新旧代码契约错位（例如系统设置保存后 maxPrice=100 开始真实生效、发布超限商品被拒；密码策略 medium 拒绝纯数字演示口令）。这些失败恰恰证明约束已生效，正确的处置是让测试与文档对齐新契约，而不是放宽校验。

2.4 Prompt 模式示例

以下提炼自本次升级中复用率最高的三类 Prompt 骨架。共同特征：把事实、约束与验收标准写进指令，把方案选择权留给受约束的生成，并把「完成」定义为可机器判定的条件。

清单 2-1 三类高频 Prompt 骨架

【模式一 · 盘点类】
对 <目录> 做 very thorough 的只读盘点。输出结构化报告，包含：
1) 接口清单（方法/路径/功能/权限）；2) 各域业务现状；
3) 安全与健壮性问题清单（附文件:行号）；4) 配置审查；
5) 对照完整平台列出缺失功能。报告将用于编写升级清单，不要给泛泛建议。

【模式二 · 实施类】
按清单编号 A2 修复：<问题描述>。约束：不引入新依赖；复用既有校验模式；
修改前先读 <文件> 全文。完成定义：mvn compile 零错误，冒烟测试无新增失败。

【模式三 · 纠偏类】
冒烟测试出现 N 项失败。先逐项归因：属于「新契约生效导致旧断言过时」的，
修正断言并在注释中写明契约依据；属于「代码缺陷」的，修复代码。

模式三的价值在于把「测试失败」从交付噪声转化为契约演进的驱动信号：本轮 26 项失败全部按此流程归因，多数归入「设置与密码策略真实生效导致旧断言过时」，其余为测试侧缺陷（如注册用例生成的手机号包含字母）并逐一修复。

2.5 阶段划分与验证闸门实录

五个阶段的进入条件、退出条件与实际执行结果如表 2-4。每个阶段退出前都执行编译验证；全量冒烟在阶段二后与全部完成后各执行一次。

表 2-4 实施阶段与验证闸门执行实录

阶段	范围	退出条件	实际结果
阶段一	A 组必修缺陷（A1 ~ A10）	编译零错误；核心鉴权链路行为符合预期	通过（期间发现并重写一处引用已删除字段的生成代码）
阶段二	E 组安全项（E1/E2/E3/E7）	编译零错误；事务注解与条件更新就位	通过；随后执行全量冒烟，暴露 26 项契约类失败并完成归因
阶段三	B 组用户端（B1 ~ B11）	编译零错误；新增接口进入 Swagger	通过；冒烟测试扩至 336 项
阶段四	C 组管理端（C1 ~ C7）	编译零错误；管理端新页面可走查	通过
阶段五	D 组体验 + F 组工程化	前端构建通过；两套测试全绿	通过（336 项冒烟 + 66 项单元测试）

2.6 防跑偏与纠偏机制

- 范围闸门：清单之外的功能想法一律记入「本轮不做」，不在实施过程中顺手实现；
- 验证闸门：见表 2-4，验证前置于交付而非事后补测；

- 独立安全检查：仓库接入的独立安全工具（Mimosa）在源码写入与提交前做实时扫描，曾两次拦截冒烟测试中的硬编码口令写法，迫使其改为环境变量常量（口令来自公开种子数据，属演示凭据）；
- 自省式重写：生成过程中若发现自己写出引用已删除字段的包装方法，立即停止追加、回读文件并整体重写该段，而不是叠加补丁；
- 对照取证：对「单测失败是否为本次升级引入」这类关键判断，用 `git worktree` 在未改动的基线提交上复跑同版本测试，取得对照证据后再下结论——实测证明 63 项失败在升级前即存在。

2.7 方法局限性与适用边界

上述方法并非普适，以下几点是本轮实测到的边界：

- 依赖高质量清单：流程的收益来自「先审查后生成」。若跳过人工审查直接让 AI 自由实施，清单的锚定作用即失效——本轮 56 项中有 4 项正是在审查环节被砍掉或降级的；
- 存量代码质量决定约束效果：结构约束之所以有效，是因为既有代码的分层与命名本身就是高质量范例；在分层混乱的仓库里，「照抄邻近实现」会把坏味道一并复制；
- 契约错位需要人工裁决：测试失败归因（新契约生效 vs 代码缺陷）无法完全自动化，本轮 26 项失败的归因结论均经人工确认；
- 上下文窗口仍是硬约束：单个会话难以容纳全部代码，依赖分级加载与任务清单锚点，代价是跨会话的上下文重建成本（见 2.2.3）。

第三章 架构设计

本章先给出总体架构、分层依赖与数据模型（3.1 ~ 3.3），再按请求路径依次说明契约层、鉴权链路 & 并发控制（3.4 ~ 3.6），然后是前端架构与接口设计规范（3.7、3.9），最后归纳对 AI 生成代码施加的约束手段（3.8）。

3.1 总体架构

系统为前后端分离结构：Vue 3 单页应用在开发期经 Vite 代理、生产期经 Nginx 访问 Spring Boot 单体服务；数据落在 MySQL 与 Redis。单体内部按业务域垂直切分模块，每个域内再做控制器/服务/持久化分层；鉴权、异常、缓存、字段自动填充、定时任务等横切能力收敛到独立的 config 与 task 包。

图 3-1 系统总体架构与请求路径



3.2 后端分层与依赖规则

每个业务域内部固定为「controller → service/serviceImpl → mapper → entity」的单向调用链，并遵守三条依赖规则：控制器不直接访问 Mapper；服务之间允许单向复用（如 admin 复用 product 的级联删除），禁止互相循环依赖；跨域数据访问一律通过对方的 Mapper 或 Service，而非直接写 SQL。各模块职责与本轮变更如下：

表 3-1 后端模块职责与本轮变更

模块	职责	本轮主要变更
user	注册、登录、资料、改密、头像	登录锁定与密码策略接入、会话时效由设置驱动、字段清空语义修正
product	商品发布/编辑/上下架/搜索/附近/评论	重新上架与彻底删除、编辑状态校验、事务补齐、评论购买校验、搜索多字段、估价开关
order / cart	下单锁定、订单状态机、购物车与收藏互转	支付与确认收货条件更新、超时自动取消、卖家可见订单详情
message	系统消息收件箱、私信与商品咨询	收件箱分页化、群发批次号、批量插入
admin	管理后台各页接口	扩展订单/评论/举报/趋势接口；删除商品复用商品模块级联逻辑
report (新增)	举报提交与处理	全新模块：实体/Mapper/服务/控制器
task (新增)	定时维护	超时订单取消、商品到期自动下架
config / utils	缓存、序列化、JWT、文件存储、请求上下文	缓存管理器显式化、序列化白名单、魔数校验、UserHolder 扩展

3.3 数据模型与索引设计

数据库共 12 张表（本轮新增 report 与 system_setting，扩展 system_message 的 batch_no 列），围绕「用户-商品-交易-消息」四条主线组织：

表 3-2 数据表清单与关键设计

表	职责	关键设计
user	账号	BCrypt 密文；role 区分管理员；status 支持禁用
category / product / product_image	分类与商品	商品状态机 1 在售 / 2 已售出 / 3 已下架；图片表首图冗余为封面
user_product_relation	收藏与购物车	同表以 relation_type 区分，唯一索引防重，互转零成本
product_order / order_item	交易	明细保存下单快照（标题/封面/价格），商品后续变更不影响历史订单
comment	评价	status 隐藏位预留；评分与内容由 DTO 校验
private_message	私信与咨询	会话键由双方用户 ID 排序拼接加商品 ID 构成，天然隔离越权
system_message	系统通知	receiver_type 群发标记 + batch_no 批次号（本轮新增）
report（新增）	举报	target_type + target_id 多态指向商品或评论，处理动作与备注留痕
system_setting（新增）	系统设置	单行表整体 JSON 覆盖；读取失败回落默认值

索引方面，product 表针对高频查询路径建立了组合索引（建表语句节选见附录 D），设计意图见表 3-3：

表 3-3 product 表索引设计与用途

索引	类型	服务的查询
uk_username（user 表）	唯一	登录与注册时的用户名等值查找，并防止重名
idx_status	普通	前台列表/搜索固定按 status=1 过滤
idx_status_category / idx_status_price	组合	分类筛选与价格区间筛选的复合条件
idx_create_time	普通	「最新发布」排序与看板按日统计
idx_longitude	普通	附近商品查询的矩形预过滤（配合 SQL 中的 FORCE INDEX，见 3.6）
uk_user_product（关联表）	唯一	(user_id, product_id, relation_type) 防止重复加购/收藏

3.4 统一契约层

- 统一返回：所有接口返回 Result{code,msg,data}，业务码集中定义于 ResultCodeEnum，各模块不得自定义魔法值；注意 HTTP 状态码恒为 200，业务失败靠 body 内的 code 区分，前端拦截器统一处理；
- 统一异常：业务错误抛 BusinessException，全局处理器兜底为固定文案，堆栈仅写入日志不外泄；
- 统一校验：入参 DTO 以 JSR-303 注解声明约束（@Pattern/@Size/@NotNull），控制器启用 @Validated，服务层只负责跨字段与状态类校验；

- 统一填充：create_time/update_time 由 MyMetaObjectHandler 自动填充，业务代码不再手工赋值。

3.5 鉴权与安全链路

本轮将「登录态解析」与「账号状态校验」合并进同一条拦截器链，消除了「禁用用户在 Token 有效期内仍可交易」的窗口期，同时使管理端角色校验不再额外查库：

图 3-2 请求鉴权链路



拦截器的放行范围分两类：完全放行（未登录不可达业务数据）与「可选鉴权」（游客可读、登录后附带个性化字段，如商品详情的联系方式与收藏状态）。具体清单见表 3-4：

表 3-4 公开路径与鉴权策略

路径	策略	说明
/user/login、/user/register、/admin/login	完全放行	登录注册入口
/product/search、/product/detail/**、/product/categories	可选鉴权	游客可浏览；登录后返回是否收藏等附加字段
GET /product/{id}/comments	方法级放行	评论列表游客可读，发表评论仍需登录（public-paths 支持「GET:路径」语法）
/uploads/**	完全放行	图片经 img 标签直接访问，不携带 Authorization 头
/public/**（新增）	完全放行	站点名称、上传限制、估价开关等前台公开配置
/health、/health/detail	完全放行	容器健康检查与部署脚本探测

3.6 并发与一致性设计

二手商品「单件唯一」的属性决定了交易的核心矛盾是并发购买。项目用「条件更新 + 影响行数判断」替代「先查后改」，关键代码形状如下：

清单 3-1 下单锁定商品与支付状态推进的条件更新（节选自 OrderServiceImpl）

```
// 下单：仅当商品仍在售时才能锁定，更新不到说明刚被别人买走
int changed = productMapper.update(null, new LambdaUpdateWrapper()
    .eq(Product::getId, product.getId())
    .eq(Product::getStatus, ProductStatusEnum.ON_SHELF.getCode())
    .set(Product::getStatus, ProductStatusEnum.SOLD.getCode()));
if (changed == 0) { throw new BusinessException("该商品刚刚被其他买家买走"); }

// 支付：仅待支付状态可推进，避免并发下重复支付或跳过状态
int changed = productOrderMapper.update(null, new LambdaUpdateWrapper()
    .eq(ProductOrder::getId, order.getId())
    .eq(ProductOrder::getStatus, OrderStatusEnum.PENDING_PAY.getCode())
    .set(ProductOrder::getStatus, OrderStatusEnum.PAID.getCode())
    .set(ProductOrder::getPayTime, LocalDateTime.now()));
if (changed == 0) { throw new BusinessException("订单状态已变化，请刷新后重试"); }
```

表 3-5 并发与一致性机制汇总

场景	机制	效果
下单锁商品	条件更新 + 事务回滚（影响行数为 0 即抛异常）	同一件商品不会被两个买家同时购得
支付 / 确认收货	同款条件更新（WHERE status=期望值），失败提示刷新重试	并发下订单状态不会重复推进或跳状态
超时未支付	定时任务逐单条件更新取消并释放商品，与买家手动取消互不冲突	30 分钟未支付的订单确定性回收
系统设置	落库 system_setting 单行表，保存即整体覆盖；读取失败回落默认值	设置真实约束业务（上传/限价/锁定/时效），且 Redis 重启不丢失
分类缓存	@Cacheable 写 Redis（TTL 10 分钟），管理端增删改时整组驱逐	高频下拉查询不落库，变更即时可见
上传安全	扩展名白名单（后台可配）+ 文件头魔数校验 + UUID 落盘	伪装成图片的可执行内容被拒绝

3.7 前端架构

前端按「布局 - 视图 - 共享组件 - 基础设施」四层组织：三个布局（MainLayout/AuthLayout/AdminLayout）承载导航与角标；视图按路由模块懒加载；本轮新增的共享组件 ProductCard 与 ApplImage 消除了商品卡片在两处页面重复实现、图片坏链无兜底两类问题。

- 状态管理：Pinia user store 是唯一状态中心（登录态归一化、未读私信/通知/购物车三个角标），Token 的持久化读写收敛在 utils/auth；
- 请求层：axios 拦截器统一注入 Token、解包 Result、401 清态跳登录、业务错误统一提示，页面 catch 中不再重复弹错；
- 契约对齐：utils/constants.js 集中维护与后端一致的枚举映射（订单 1~4、商品 1~3），本轮修正了过时的「7 状态」订单枚举并新增状态映射表供各页复用；
- 移动端：前台顶栏与后台侧边栏补齐汉堡菜单，私信中心双栏在窄屏改为上下堆叠。

「待补充：关键页面截图（首页、商品详情、管理看板），便于读者对照理解。」

3.8 对 AI 生成代码的约束手段

架构约束的本质是压缩新代码的自由发挥空间。本项目把约束分为四层，强度递增：

表 3-6 对 AI 生成代码的四层约束

层级	手段	实例
结构约束	沿用既有域分包与分层命名，新模块复用成熟模块的骨架（report 模块完全参照 message 模块结构）	新增 report、task 两个包时无架构分歧
契约约束	枚举集中定义、DTO 校验注解、接口注释与 Swagger 注解齐全、Result 统一返回	全部新接口自动纳入拦截器保护，无一例外
流程约束	编辑前必读文件全文；每阶段编译；关键判断以对照实验取证	约 900 行的服务类先通读再修改，避免盲改
验证约束	冒烟测试将「设置真实生效」「评论需购买」等新契约固化为断言，回归不通过不得交付	336 项断言即架构的行为规范

一个典型的纠偏案例：扩展管理端时，模型曾生成一组引用已删除字段的包装方法；发现后没有叠加补丁，而是恢复必要的 Mapper 注入并将该段整体重写为直接调用。约束体系的价值不在于杜绝错误，而在于让错误被结构化的流程快速捕获并纠正。

3.9 接口设计规范

为保证 AI 生成的几十个接口风格一致，升级前明确了以下规范，并作为评审清单使用：

- URL：资源用名词复数（/users、/products），动作用动词后缀（/{id}/status、/{orderNo}/pay、/{id}/reshelf）；删除分「软下架」（DELETE /product/{id}）与「物理清除」（DELETE /product/{id}/purge）两类并显式区分；
- 参数：列表接口统一 page/pageSize 分页（服务端 maxLimit=200 兜底）；身份类参数一律取自登录态（UserHolder），禁止前端传入 userId；
- 状态语义：业务码复用 ResultCodeEnum（401 未登录、403 无权限、400 参数错误、501 系统内部错误）；禁止新增与既有语义重叠的码；
- 文档：每个端点必须有 @ApiOperation 说明与权限标注；新增公开路径必须同步登记到表 3-4 对应的策略清单，避免「忘了放行」与「忘了保护」两类事故。

第四章 可复现性论证

4.1 复现性声明

在满足 README 所列环境要求 (JDK 8+、Node 18+、MySQL 8、Redis 7) 的前提下，任何第三方从仓库零起步，可在 10 分钟内完成系统复现，并以单元测试 66 项、冒烟测试 336 项断言全部通过验证行为一致。复现性依赖以下四项机制：环境即代码、配置有默认值且可覆盖、行为有自动化断言、无外部付费服务依赖。

4.2 环境与依赖锁定

- 后端依赖由 pom.xml 锁定 (Spring Boot 2.7.12 parent 统一管理传递依赖版本)，构建产物为 second-hand-market-1.0.0.jar；
- 前端依赖由 package.json 与 package-lock.json 双重锁定，npm install 结果跨机器一致；
- 运行时要求宽松 (JDK 8+ 即可编译)，仓库 .tools/ 目录附带本机专用 Maven 3.9.16，消除工具链差异；
- 仅有的外部依赖是本地部署的 MySQL 与 Redis，不依赖任何云服务或付费接口。

4.3 数据与配置即代码

- 建库即重建：backend/scripts/01-schema.sql 单脚本创建全部 12 张表并写入种子数据 (3 个账号、2 件在售商品、8 个分类)，无手工步骤；
- 增量升级可重放：存量库执行 02-upgrade.sql 一次完成本轮表结构演进 (report、system_setting、batch_no)；
- 配置零改动可运行：application.yml 全部环境相关项写成 \${VAR:默认值} 形式，默认值即本地开发可用值，部署时以环境变量覆盖 (完整清单见附录 A，关键段节选见附录 D)；
- 一键脚本：scripts/ 目录提供依赖拉起、建库、后端重建、一键启动四个批处理脚本，复现流程不依赖操作者记忆。

环境变量的典型用法 (完整模板见 README)：

清单 4-1 环境变量注入示例

```
# Linux / macOS
export DB_PASSWORD='your-strong-password'
export JWT_SECRET='a-very-long-random-secret-at-least-32-bytes'
java -jar backend/target/second-hand-market-1.0.0.jar

# Windows PowerShell
$env:DB_PASSWORD='your-strong-password'
$env:JWT_SECRET='a-very-long-random-secret-at-least-32-bytes'
java -jar backend\target\second-hand-market-1.0.0.jar
```

4.4 行为可验证：把复现从「能跑起来」变为「断言全绿」

复现成功的判定不依赖人工浏览页面，而以两组自动化套件为准。冒烟测试 `tests/api_smoke_test.py` 共 336 项断言，按 31 个业务分节组织（纯 Python 标准库实现，无第三方依赖，直接以 ``python tests/api_smoke_test.py [base_url]`` 运行，全程约 1 分钟）；单元测试 66 项由 `mvn test` 执行。任何契约回归都会在分钟级暴露。分节断言分布见附录 E 表 E-1。

冒烟测试设计了三项自愈机制以保证「可反复运行」：其一，脚本启停时按标题前缀清理历史测试数据（商品、分类、系统消息），断言不依赖残留状态；其二，注册类用例使用随机后缀账号，避免唯一键冲突；其三，脚本启动时由管理员将演示账号口令重置为种子值，防止上一轮中途失败导致的口令漂移。

4.5 性能基准实测

为给「性能」这一维度留下可复现的基线，使用仓库自带压测工具 `tests/stress_test.js` 对三个代表性接口做了实测。测试条件：10 并发线程 × 每线程 50 请求（共 500 请求/轮），本机环境（Windows 11、JDK 22、MySQL 8 与 Redis 7 同机部署），数据量为种子数据量级。结果如表 4-1：

表 4-1 接口性能基准实测（10 并发 × 500 请求，2026-10）

场景	成功率	平均响应	中位响应	最大响应	吞吐量
GET /health (Redis 探活, 无 SQL)	100%	211 ms	235 ms	255 ms	约 1323 RPS
GET /product/search (MySQL 分页查询)	100%	455 ms	450 ms	635 ms	约 700 RPS
POST /user/login (BCrypt 校验 + 缓存写入)	100%	2624 ms	2731 ms	3573 ms	约 135 RPS

三点口径说明：其一，响应时间由 Node 压测客户端测量，包含客户端调度开销（同一接口用 `curl` 单发实测服务端耗时为 `health` 约 2 ms、`search` 约 16 ms），绝对数值仅作同环境横向对比参考；其二，`login` 明显偏慢属于预期行为——BCrypt 是故意设计为 CPU 密集的口令散列（代价因子 10），10 并发下排队显著，这正是「防止口令被快速暴力枚举」的代价换来的安全收益；其三，种子数据量级下数据库尚未成为瓶颈，压测不能外推为生产容量结论。

此次实测过程本身是一次有代表性的工程教训：第一轮压测曾得到「`search` 平均响应 51 万毫秒、登录 0% 成功」的荒谬数据。逐层排查后发现两个层面的问题，工具侧缺陷与修复见表 4-2；环境侧则是 MySQL 进程已停止，而本项目 HTTP 状态码恒为 200、业务失败藏在 `body` 的 `code` 字段里，压测工具按 HTTP 状态码统计出「100% 成功」——失败被完全掩盖。修复工具、重启 MySQL 后重测才得到表 4-1 的有效数据。

表 4-2 压测工具缺陷清单与修复（`tests/stress_test.js`）

缺陷	现象	修复
<code>errorCounts</code> 跨作用域引用	汇总阶段抛 <code>ReferenceError</code> ，统计报告无法输出	在 <code>analyzeResults</code> 内基于 <code>results</code> 重新统计 <code>r.error</code>
<code>startTime</code> 跨作用域引用	总耗时与 RPS 无法计算	改为函数入参传入
<code>args</code> 跨作用域引用	结果文件写入前崩溃	改为函数入参传入

缺陷	现象	修复
URL 查询串被丢弃	options.path 只取 pathname，带参压测实际打在无参接口上	path 改为 pathname + search 拼接
POST 缺 Content-Type 头	带 JSON payload 的请求可能被服务端以 415 拒绝	payload 非空时自动补 application/json

该案例的启示：其一，「HTTP 恒 200」契约会掩盖业务失败，自动化验证必须解析业务码；其二，压测前应先确认依赖服务存活（本例 /health 不检查数据库，未能起到探活作用）；其三，测试工具自身也需要被测试——三处作用域缺陷在首次真实运行时才暴露。

4.6 已知非确定性因素与影响分析

表 4-3 非确定性因素与控制手段

因素	来源	影响范围	控制手段
AI 估价随机波动	估价规则含 ± 200 元扰动，模拟 AI 不确定性	仅估价建议数值，不影响发布	单元测试断言区间而非精确值
时间戳	create_time、支付时间等取运行时刻	展示与排序，不影响业务	测试断言不比较绝对时间
并发购买结果	两人同时购买同一件商品，胜者取决于竞争顺序	订单归属的不确定性	条件更新保证「恰好一人成功」这一确定性
登录锁定计数	失败次数存 Redis，重置环境即清零	锁定时长内的登录尝试	冒烟测试不依赖锁定状态

4.7 复现演练记录

本轮升级完成后执行过一次完整演练，过程与结果如下，可作为复现指南的实测基线：

表 4-4 复现演练记录

步骤	内容	结果
环境	Windows 11 / JDK 22 / Node 22 / MySQL 8 / Redis 7（本机服务）	就绪
建库	对既有库执行 02-upgrade.sql，确认 12 张表与 batch_no 列	通过
构建	mvn -DskipTests package 生成 jar；前端 npm run build 通过并完成分包	通过
启动	java -jar 启动后端，/api/health 返回 code=200	通过
单元测试	mvn test: Tests run 66, Failures 0, Errors 0	通过
冒烟测试	python tests/api_smoke_test.py: 通过 336，失败 0	通过
行为抽检	登录 Token 有效期随「会话超时」设置变为 12 小时，验证设置真实生效	通过

4.8 复现性的局限

如实声明三类局限：其一，性能数据（4.5 节）依赖具体机器的磁盘与负载，仅提供量级参考，换机复现时吞吐与延迟不保证一致；其二，HTTP 状态码恒为 200 的契约使「以 HTTP 状态判断健康」的通用工具失效——本轮压测即被此误导，自动化验证必须解析 body 中的业务码，该契约本身已列入后续演进项（见 3.9 与第五章）；其三，冒烟测试的「可重复运行」依赖测试自愈逻辑（按标题前缀清理历史数据、随机后缀账号、管理员重置演示口令），若种子数据被手工修改，部分断言可能失效。

第五章 总结与展望

本轮升级验证了一条可复用的 AI 辅助改造路径：以结构化盘点建立共同事实，以经人工审查的编号清单锚定意图，以分阶段验证闸门控制风险，最终以自动化断言交付可复现的结果。56 项清单中 52 项落地，4 项按审查约定延后并记录了理由；两套测试从部分失真恢复到全绿；安全工具与测试套件在过程中各自拦截了真实问题（前者拦截硬编码凭据写法，后者暴露设置真实生效带来的契约错位），说明「约束 + 验证」的组合比「生成后人工审读」更可靠。

后续路线建议按三个方向推进：其一，接入真实支付网关与 WebSocket 推送（本轮明确延后的两项，接口形状已为其预留），并评估将「HTTP 恒 200」契约改为语义化状态码；其二，将冒烟测试纳入 CI，在每次提交时自动执行，同时修复压测工具与 CI 的集成；其三，补齐本轮标注为「待补充」的项目背景与界面截图，使文档自治程度进一步提升。

附录 A 环境变量表

表 A-1 可注入环境变量清单

变量	默认值	说明	部署必改
DB_HOST / DB_PORT	localhost / 3306	MySQL 地址	—
DB_NAME	secondhand_market	库名，与建表脚本一致	—
DB_USERNAME / DB_PASSWORD	root / 1234	数据库凭据	密码必改
REDIS_HOST / REDIS_PORT	localhost / 6379	Redis 地址	—
JWT_SECRET	内置默认串	HS256 签名密钥，不少于 32 字节	必改
JWT_EXPIRATION	604800000	Token 默认有效期（毫秒），后台安全设置可覆盖	—
FILE_UPLOAD_DIR	./uploads	上传落盘目录	建议挂独立数据卷

附录 B 接口清单摘要

表 B-1 按模块归纳的 REST 端点

模块	代表性端点	权限
user	POST /user/register · POST /user/login · GET/PUT /user/info · PUT /user/password · POST /user/avatar	注册登录免鉴权，其余需登录
product	POST /product/add · GET /product/search · GET /product/detail/{id} · POST /product/nearby · POST /product/estimate · PUT/DELETE /product/{id} · POST /product/{id}/reshelf · DELETE /product/{id}/purge · GET/POST /product/{id}/comments · POST /product/upload	搜索/详情/分类/评论读公开，其余需登录
report (新增)	POST /report	需登录
order	POST /order/create · GET /order/list · GET /order/sold · GET /order/{orderNo} · POST /order/{orderNo}/pay cancel confirm	需登录
cart	POST/DELETE /cart/{productId} · GET /cart/list · GET /cart/count · 购物车收藏互转	需登录
message	GET /message/list · POST /message/{id}/read · POST /message/all/read · GET /message/unread/count · POST /message/private/send · GET /message/private/chat · GET /message/private/conversations	需登录
admin	GET /admin/dashboard · /admin/stats/trend · /admin/stats/category · /admin/orders · /admin/comments · /admin/reports · /admin/users · /admin/products · /admin/categories · /admin/messages · GET/PUT /admin/settings · POST /admin/upload · /admin/export/*	需管理员
public (新增)	GET /public/settings	免鉴权

附录 C 常见问题

- 接口统一返回 code=501「系统内部错误」：MySQL 或 Redis 未启动。运行 scripts/start-dev-services.bat 后无需重启后端，应用会自动重连；注意 /health 不检查数据库，即使 MySQL 已停它仍返回 200，应以业务接口的 body 业务码为准；
- 报 Table secondhand_market.user doesn't exist：库未初始化，执行 scripts/init-database.bat 或手动导入 backend/scripts/01-schema.sql；
- 老库启动报 Unknown column batch_no：执行一次 backend/scripts/02-upgrade.sql；
- 注册报「密码强度不足」：后台安全设置默认 medium（须同时包含字母与数字），演示可临时调为 low；
- 发布商品报「超过平台上限」：后台交易设置的价格上限真实生效，按需调整；
- 图片上传失败：确认上传目录可写、文件为真实图片（服务端校验文件头），且单张不超过后台设置的大小上限。

附录 D 配置样例

以下为 application.yml 中与复现直接相关的关键段节选（完整文件见仓库）。所有 \${VAR:默认值} 均可被同名环境变量覆盖：

清单 D-1 application.yml 关键段（节选）

```
server:
port: 8080
servlet:
context-path: /api # 所有接口以 /api 开头
spring:
datasource:
url: jdbc:mysql://${DB_HOST:localhost}:${DB_PORT:3306}/${DB_NAME:secondhand_market}?useSSL=false&serverTimez
one;=Asia/Shanghai
username: ${DB_USERNAME:root}
password: ${DB_PASSWORD:1234}
servlet:
multipart:
max-file-size: 5MB # 必须大于 FileStorageUtil 的单文件校验值
max-request-size: 10MB
jwt:
header: Authorization
secret: "${JWT_SECRET:SecondHandMarket@Jwt$Secret$Key$2026$Competition}"
expiration: ${JWT_EXPIRATION:604800000} # 7 天
public-paths: /user/login,/user/register,/admin/login,/health/**,/product/search,
/product/detail/**,/product/categories,GET:/product/*/comments,
/uploads/**,/public/**
file:
upload-dir: ./uploads
access-prefix: /api/uploads
cors:
origin-patterns: "*" # 生产环境必须收敛为具体域名
```

Docker Compose 部署时，backend 服务的 environment 段可直接注入同名变量：

清单 D-2 docker-compose.yml 环境变量注入（节选）

```
services:
backend:
environment:
DB_HOST: mysql
DB_PASSWORD: ${DB_PASSWORD:-1234}
JWT_SECRET: ${JWT_SECRET:-change-me-in-production}
REDIS_HOST: redis
```

依赖版本的关键锚点（完整清单见 pom.xml 与 package-lock.json）：Spring Boot 2.7.12、MyBatis-Plus 3.5.3.1、JJWT 0.11.5、Knife4j 4.1.0、mysql-connector-j 8.0.33、Vue ^3.4、

Element Plus ^2.4、ECharts ^5.5、Axios ^1.6。

附录 E 测试用例分布

冒烟测试（tests/api_smoke_test.py）按 31 个业务分节组织，共 336 项断言，可重复执行（自带历史数据清理与演示口令自愈逻辑）。分节分布如下：

表 E-1 冒烟测试分节断言分布（共 336 项）

分节	断言数	分节	断言数	分节	断言数
1 健康检查	1	2 鉴权拦截	1	3 管理员登录	3
4 普通用户越权	3	5 数据概览	7	6 用户管理	14
7 商品管理	8	8 分类管理	9	9 系统消息	11
10 系统设置	11	11 数据导出	7	12 商品分类	2
13 发布商品	9	14 商品搜索	14	15 商品详情	13
16 删除商品（管理端）	5	16.5 删除系统消息	6	17 图片上传	5
18 扩展字段持久化	12	19 收藏	12	20 我的发布/编辑/下架	15
21 个人资料	12	22 修改密码	9	23 头像上传	6
24 商品评价	21	25 商品多图	9	26 上传大小限制	3
27 私信与商品咨询	32	28 购物车	18	29 订单与模拟支付	42
30 升级新增能力	16	—	—	—	—

单元测试共 66 项，分布于 6 个测试类，覆盖点见表 E-2：

表 E-2 单元测试类分布（共 66 项）

测试类	用例数	覆盖点
ProductServiceFullTest	22	附近商品查询的全场景：坐标边界、半径、分页、异常参数
ProductAIServiceFullTest	17	估价规则区间、品牌/型号/长标题溢价、异常参数、并发
ProductAIServiceImplTest	12	估价成功路径与非法参数的失败返回契约
ProductServiceImplTest	10	附近查询核心路径（LBS）
ProductAIServiceSimpleTest	3	Spring 集成：真实缓存命中验证
ProductServiceSimpleTest	2	Spring 集成冒烟

附录 F 术语表

表 F-1 术语表

术语	定义
优化清单 / A ~ F 编号	本轮升级的工作分解：56 项条目按 A 必修缺陷、B 用户端、C 管理员端、D 前端体验、E 安全健壮性、F 工程化六组编号，正文以「A2」「C8」形式引用
未生效功能	界面或文档声称存在、但后端无实现或实现未接线的功能（如排序只改前端数组不调接口），本轮修复的重点类别之一
条件更新	UPDATE 语句在 WHERE 中携带状态前置条件（如 status=在售），以影响行数判断是否生效；本项目用于下单锁商品与订单状态推进，替代「先查后改」消除竞态
可选鉴权	拦截器策略：游客可访问公开路径，但携带有效 Token 时仍解析登录态，使「是否收藏」等个性化字段对登录用户可见
魔数校验	按文件头二进制签名（如 PNG 的 89 50 4E 47）识别真实文件类型，防止伪装扩展名上传可执行内容
BCrypt	自带盐值与可调代价因子的自适应口令散列算法，校验刻意耗时以对抗暴力枚举
登录锁定	同一用户名连续失败达阈值（默认 5 次）后，拒绝登录一段时长（默认 30 分钟），计数与锁定期均存 Redis
批次号 (batch_no)	群发系统消息时同一次群发的全部记录共享的 UUID，用于管理端聚合展示与整组删除
会话键 (conversation_key)	由「较小用户 ID-较大用户 ID-商品 ID」拼接的私信会话标识，只有会话双方能算出同一键，天然防越权
幽灵依赖	未在 package.json 声明、靠传递依赖偶然可用的 npm 包；升级后已显式声明
N+1 查询	列表查询后对每条记录再发起一次关联查询的反模式；本项目以 selectBatchIds 批量回查规避
跨作用域引用	JavaScript 闭包中引用了不存在的上层变量导致 ReferenceError；本轮在压测工具中修复三处同类缺陷
乐观更新	前端在请求成功后先修改本地状态再等待刷新的交互方式，用于私信发送与收藏计数
幂等加购	重复加入购物车不报错也不产生重复记录，由 (user_id, product_id, relation_type) 唯一索引保证
种子数据	建表脚本自带的演示数据：admin/admin123（管理员）、testuser1、testuser2（口令 123456）及 2 件在售商品、8 个分类

附录 G 端到端案例：一件商品从发布到售出

本案例串起平台最核心的业务闭环，所有步骤均有冒烟断言覆盖（对应表 E-1 的 13、14、28、29、24、30 各分节）。设卖家为 testuser1，买家为 testuser2：

表 G-1 交易闭环步骤与状态变化

步骤	操作 (API)	商品状态	订单状态	备注
1	卖家发布：POST /product/add (价格 100 元，线上交易)	1 在售	—	返回 productId；服务端校验分类、成色、价格上限
2	买家搜索：GET /product/search?key word=xxx	1 在售	—	关键词命中标题/描述/品牌/型号，仅返回在售商品
3	买家查看详情：GET /product/detail/{id}，浏览量 +1	1 在售	—	联系电话仅登录用户可见
4	买家加购：POST /cart/{productId}	1 在售	—	幂等；自己的商品会被拒绝
5	买家咨询：POST /message/private/send (带 productId)	1 在售	—	会话按双方 ID + 商品 ID 隔离
6	买家下单：POST /order/create (remark 留言)	1 → 2 已售出	1 待支付	条件更新锁定商品；订单明细存标题/价格快照；商品移出购物车
7	另一买家重复下单：POST /order/create	2 已售出	—	被拒：「刚刚被其他买家买走」
8	买家支付：POST /order/{orderNo}/pay	2 已售出	1 → 2 已支付	模拟支付，不产生真实扣款；重复支付被拒
9	买家确认收货：POST /order/{orderNo}/confirm	2 已售出	2 → 4 已完成	仅已支付状态可确认
10	买家评价：POST /product/{id}/comments (评分 5)	2 已售出	4 已完成	仅购买过的用户可评；评论带「已验证购买」标识
11	卖家查看：GET /order/{orderNo}	2 已售出	4 已完成	卖家可查看含自己商品订单的详情
12	管理员导出：GET /admin/export/orders	—	—	CSV 含订单明细快照，公式注入已防护

对照分支：若第 8 步买家 30 分钟内未支付，定时任务（每 10 分钟扫描）会将订单置为「已取消」并把商品释放回「在售」（状态 2 → 1），买家手动取消走同一释放逻辑；若卖家在第 6 步前下架商品（状态 3），下单会被「已下架」拒绝。取消与超时分支分别由冒烟测试分节 29 与后台定时任务覆盖。

附录 H 参考资料

- [1] VMware Tanzu. Spring Boot Reference Documentation (2.7.12).
<https://docs.spring.io/spring-boot/docs/2.7.12/reference/html/>
- [2] 苞米豆团队. MyBatis-Plus 官方文档 (3.5.x) . <https://baomidou.com/>
- [3] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT): RFC 7519. IETF, 2015.
<https://www.rfc-editor.org/rfc/rfc7519>
- [4] Provos N, Mazières D. A Future-Adaptable Password Scheme. Proceedings of the USENIX Annual Technical Conference (FREENIX Track), 1999. (BCrypt 原始论文)
- [5] OWASP Foundation. OWASP Cheat Sheet Series (含 Authentication、SQL Injection Prevention 等篇) .
<https://cheatsheetseries.owasp.org/>
- [6] Vue.js 团队. Vue.js 3 官方文档. <https://cn.vuejs.org/>
- [7] Element Plus 团队. Element Plus 组件库文档. <https://element-plus.org/zh-CN/>
- [8] Apache Software Foundation. Apache ECharts 文档 (5.x) . <https://echarts.apache.org/zh/>
- [9] Sinnott R W. Virtues of the Haversine. Sky and Telescope, 1984, 68(2): 159. (附近商品检索采用的球面距离公式)
- [10] MDN Web Docs. Geolocation API. https://developer.mozilla.org/zh-CN/docs/Web/API/Geolocation_API
(发布页与附近商品使用的浏览器定位)